

RISC-V REFERENCE SOC TAPEOUT PROGRAM:

The main objective of this week was to **establish a complete digital design flow using open-source tools** and gain **hands-on experience in RTL design, simulation, verification, and synthesis**. This included understanding the end-to-end process from RTL coding to waveform analysis and gate-level synthesis, which is essential for SoC development and eventual tapeout.

1. Week 1 Highlights

1. RTL Design

- Developed fundamental digital modules using Verilog to understand hardware design at the **register-transfer level (RTL)**.
- Created **testbenches** to verify module functionality. The testbenches applied multiple input vectors, captured outputs, and compared them against expected results to ensure correctness.
- Learned the importance of writing **modular and reusable RTL code**, which simplifies simulation and synthesis.

2. Simulation with Icarus Verilog

- Compiled Verilog modules and corresponding testbenches into a **simulation executable** using Icarus Verilog.
- Ran simulations to observe **design behavior under different scenarios**, including edge cases, to confirm functionality.
- Understood how simulation helps identify **logical errors or misbehaviors** before hardware synthesis.

3. Waveform Analysis with GTKWave

- Used GTKWave to visualize **signal transitions, timing diagrams, and logic states** during simulation.
- Analyzed waveforms to confirm the **correct operation of modules** and to debug incorrect outputs.
- Gained experience in interpreting **timing relationships** between signals, which is critical for sequential logic designs.

4. Logic Synthesis with Yosys

- Transformed Verilog RTL into a **gate-level netlist** mapped to a **standard cell library**.
- Optimized logic using Yosys commands such as synth and abc to reduce gate count and improve performance.

- Prepared the synthesized netlist for **further verification** or tapeout, integrating it into an open-source EDA toolchain.

2. Icarus Verilog (iverilog) – Compile and Simulate Verilog

Purpose: Icarus Verilog compiles Verilog modules and testbenches into an executable simulation program that allows verification of functionality.

Steps:

1. Open a terminal in the folder containing your Verilog files.
2. Compile Verilog design and testbench:
iverilog FILE_NAME.v FILE_NAME_TB.v
3. Execute the simulation:
./a.out
4. Review simulation outputs to verify that all test cases pass.

Learning Outcome: This step provides confidence that the RTL design behaves correctly before moving to synthesis.

3. GTKWave – Waveform Visualization

Purpose: GTKWave allows visualization of **signal transitions** over time, enabling verification of sequential and combinational logic behavior.

Steps:

1. Ensure simulation generates a waveform file (VCD format) using Icarus Verilog.
2. Open GTKWave to inspect the waveform:
gtkwave filename.vcd
3. Analyze signals, timing relationships, and identify any discrepancies with expected behavior.

Learning Outcome: Visual inspection of waveforms is crucial for debugging and confirming correct logic operation.

4. Yosys – Logic Synthesis

Purpose: Yosys converts **RTL designs into gate-level netlists**, which are ready for **verification, optimization, and tapeout**.

1. RTL to Gate-Level Synthesis

- Converts Verilog RTL into a **gate-level netlist**.
- Maps design to standard cells from libraries such as **SkyWater 130nm**.

2. Design Optimization and Verification

- Optimizes logic using commands like `synth` and `abc` to reduce area and improve timing.
- Generates **synthesized Verilog and netlists** for further steps in the design flow.

3. Integration with Open-Source EDA Flow

- Works seamlessly with Icarus Verilog and GTKWave.
- Provides a complete open-source path for RTL to gate-level synthesis, essential for tapeout or FPGA implementation.

4.1 Step-by-Step Yosys Commands

1. Open Yosys shell:

```
yosys
```

2. Read Verilog design:

```
read_verilog FILE_NAME.v
```

3. Read standard cell library using environment variable:

```
read_liberty -lib $SKY130_LIB
```

4. Synthesize top module:

```
synth -top FILE_NAME
```

5. Write synthesized Verilog netlist:

```
write_verilog synthesized.v
```

6. Optimize logic using ABC:

```
abc -liberty $SKY130_LIB
```

7. Exit Yosys:

```
exit
```

Learning Outcome: This process ensures that the design is **optimized for area, timing, and power**, and is ready for the next stage in the SoC flow.

5. Summary and Achievements

By the end of Week 1, I successfully:

1. Set up a complete **digital design flow** using open-source tools.
2. Developed, simulated, and verified **RTL modules**.
3. Visualized signal behavior using GTKWave and debugged designs efficiently.
4. Generated a **synthesized gate-level netlist** using Yosys, ready for further verification or tapeout.
5. Gained foundational skills essential for **SoC design, verification, and tapeout preparation**.

Project: good_mux.v – 2-to-1 Multiplexer

Demonstrates RTL design, simulation, waveform viewing, and synthesis

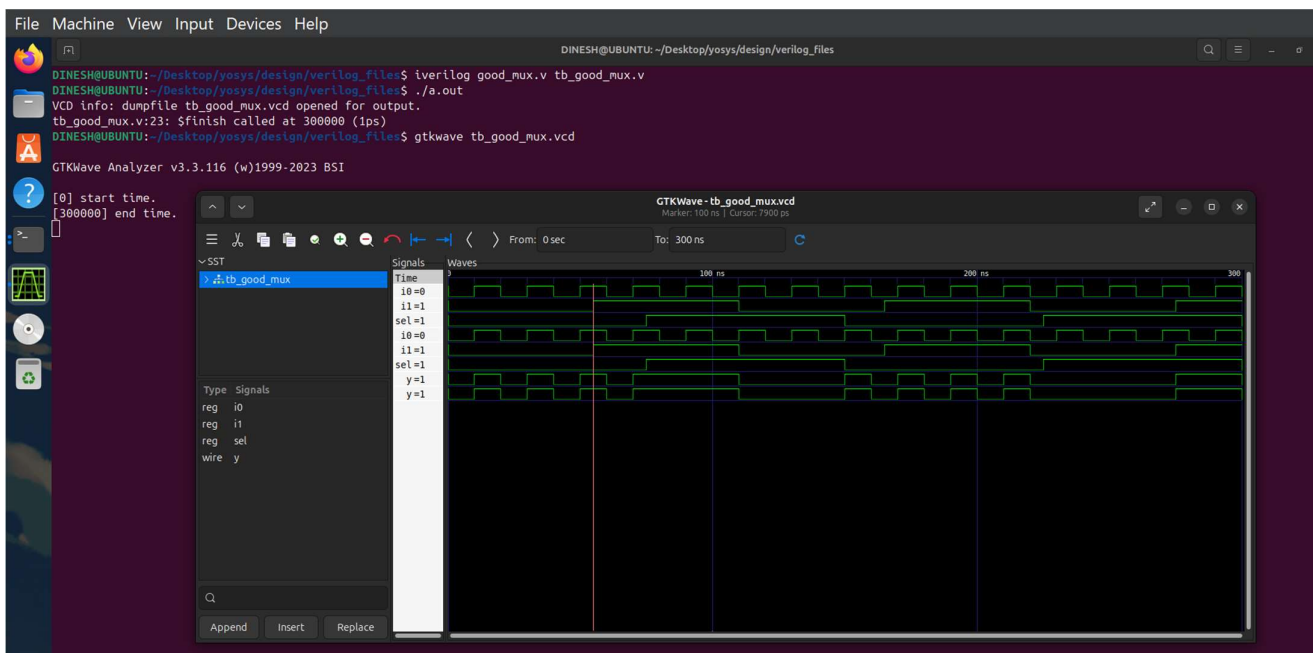
1 Compile Verilog design and testbench using Icarus Verilog

```
iverilog good_mux.v tb_good_mux.v
```

```
DINESH@UBUNTU: ~/Desktop/yosys/design/verilog_files$ iverilog good_mux.v tb_good_mux.v
DINESH@UBUNTU: ~/Desktop/yosys/design/verilog_files$ ./a.out
VCD info: dumpfile tb_good_mux.vcd opened for output.
tb_good_mux.v:23: $finish called at 300000 (1ps)
DINESH@UBUNTU: ~/Desktop/yosys/design/verilog_files$
```

2 View waveform in GTKWave

```
gtkwave good_mux.vcd
```



3 Run Yosys for logic synthesis

```
yosys << EOF
```

```
# Read the Verilog design
```

```
read_verilog good_mux.v
```

```
# Load the standard cell library
```

```
read_liberty -lib $SKY130_LIB
```

```
# Synthesize the top module
```

```
synth -top good_mux
```

```
# Write synthesized gate-level netlist
```

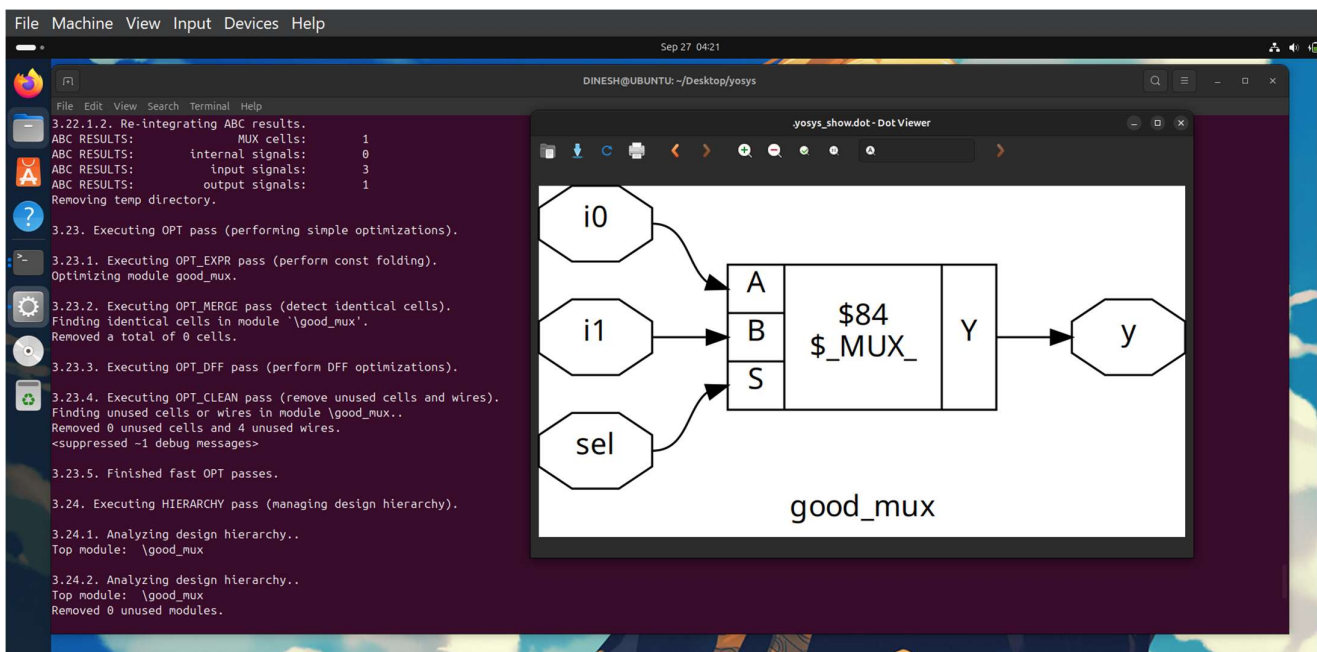
```
write_verilog good_mux_synth.v
```

```
# Optimize logic using ABC
```

```
abc -liberty \SSKY130_LIB
```

```
# Exit Yosys
```

```
exit
```



End of good_mux.v workflow

☀ Summary and Achievements:

- By the end of Week 1, I successfully:
- Set up a complete digital design flow using open-source tools.
- Developed, simulated, and verified RTL modules.
- Visualized signal behavior using GTKWave and debugged designs efficiently.
- Generated a synthesized gate-level netlist using Yosys, ready for further verification.

LAB: good_mux.v

The screenshot shows the synthesis process for the `good_mux.v` module. The terminal window displays the following output:

```

3.22.1.2. Re-integrating ABC results.
ABC RESULTS:      MUX cells:      1
ABC RESULTS:      internal signals: 0
ABC RESULTS:      input signals:   3
ABC RESULTS:      output signals:  1
Removing temp directory.

3.23. Executing OPT pass (performing simple optimizations).

3.23.1. Executing OPT_EXPR pass (perform const folding).
Optimizing module good_mux.

3.23.2. Executing OPT_MERGE pass (detect identical cells).
Finding identical cells in module 'good_mux'.
Removed a total of 0 cells.

3.23.3. Executing OPT_DFF pass (perform DFF optimizations).

3.23.4. Executing OPT_CLEAN pass (remove unused cells and wires).
Finding unused cells or wires in module 'good_mux'.
Removed 0 unused cells and 4 unused wires.
<suppressed -1 debug messages>

3.23.5. Finished fast OPT passes.

3.24. Executing HIERARCHY pass (managing design hierarchy).

3.24.1. Analyzing design hierarchy..
Top module: 'good_mux'

3.24.2. Analyzing design hierarchy..
Top module: 'good_mux'
Removed 0 unused modules.

```

The GUI window shows a logic diagram of the `good_mux` module. It has three inputs: `i0`, `i1`, and `sel`. The `sel` input is connected to a 3-to-1 multiplexer block labeled `$84 $MUX_`. The outputs of the multiplexer are `A`, `B`, and `S`. The output `A` is connected to the output `y`. The output `B` is connected to the output `y`. The output `S` is connected to the output `y`.

LAB: MULTIPLE_MODULES

The screenshot shows the synthesis process for the `multiple_modules.v` module. The terminal window displays the following output:

```

ABC: + &fraig -x
ABC: + &put
ABC: + scorr
ABC: Warning: The network is combinational (run
ABC: + dc2
ABC: + dretlme
ABC: + strash
ABC: + &get -n
ABC: + &dch -f
ABC: + &nf
ABC: + &put
ABC: + write_blif <abc-temp-dir>/output.blif

5.3.2. Re-integrating ABC results.
ABC RESULTS:  sky130_fd_sc_hd_or2_0 cells:  0
ABC RESULTS:  internal signals:            0
ABC RESULTS:  input signals:               2
ABC RESULTS:  output signals:              1
Removing temp directory.

yosys> show

6. Generating Graphviz representation of design.
ERROR: For Formats different than 'ps' or 'dot'

yosys> show multiple_modules

7. Generating Graphviz representation of design.
Writing dot description to '/home/DINESH/.yosys_
Dumping module multiple_modules to page 1.
Exec: { test -f '/home/DINESH/.yosys_show.dot.plt
'/home/DINESH/.yosys_show.dot.pid' &

```

The GUI window shows a logic diagram of the `multiple_modules` module. It has two inputs: `a` and `b`. The inputs `a` and `b` are connected to a sub-module `u1` (labeled `sub_module1`). The output of `u1` is connected to a net `net1`. The net `net1` is connected to a sub-module `u2` (labeled `sub_module2`). The output of `u2` is connected to the output `y`. There is also an input `c` connected to the output `y`.

LAB: INCOMP_IF

File Machine View Input Devices Help

Sep 27 06:06

DINESH@UBUNTU: ~/Desktop/yosys

unnapped dff cell: \$_DFFSR_PPP_

5.1. Executing DFFLEGALIZE pass (convert FFs to types supported by the target).
Mapping DFF cells in module '\incomp_if':

```
yosys> abc -liberty lib/sky130_fd_sc_hd_tt_025C_1v80.lib
```

6. Executing ABC pass (technology mapping using ABC).

6.1. Extracting gate netlist of module '\incomp_if' to '<abc-temp-dir>/input.blif'..
Extracted 0 gates and 0 wires to a netlist network with 0 inputs and 0 outputs.
Don't call ABC as there is nothing to map.
Removing temp directory.

```
yosys> show
```

7. Generating Graphviz representation of design.
Writing dot description to '/home/DINESH/.yosys_show.dot'.
Dumping module incomp_if to page 1.
Exec: { test -f '/home/DINESH/.yosys_show.dot.pid' && fuser -s '/home/DINESH/.yosys_show.dot.pid' }

```
yosys> show
```

8. Generating Graphviz representation of design.
Writing dot description to '/home/DINESH/.yosys_show.dot'.
Dumping module incomp_if to page 1.
Exec: { test -f '/home/DINESH/.yosys_show.dot.pid' && fuser -s '/home/DINESH/.yosys_show.dot.pid' }

```
yosys> /usr/lib/python3/dist-packages/xdot/ui/elements.py:174: UserWarning: Font family not supported. Using 'sans-serif' as fallback.
```

incomp_if

LAB: INCOMP_IF2

File Machine View Input Devices Help

Sep 27 06:11

DINESH@UBUNTU: ~/Desktop/yosys

ABC: Warning: Detected 9 multi-output gates (for example, "sky130_fd_sc_hd_mux2_1").

```
ABC: + strash
ABC: + &get -n
ABC: + &fraig -x
ABC: + &put
ABC: + scorr
ABC: Warning: The network is combinational (run "fraig" or "fraig -x").
ABC: + dc2
ABC: + dretime
ABC: + strash
ABC: + &get -n
ABC: + &dch -f
ABC: + &nf
ABC: + &put
ABC: + write_blif <abc-temp-dir>/output.blif
```

6.1.2. Re-integrating ABC results.

```
ABC RESULTS: sky130_fd_sc_hd_mux2_1 cells: 1
ABC RESULTS: sky130_fd_sc_hd_nor2_1 cells: 1
ABC RESULTS: internal signals: 0
ABC RESULTS: input signals: 4
ABC RESULTS: output signals: 2
```

Removing temp directory.

```
yosys> show
```

7. Generating Graphviz representation of design.
Writing dot description to '/home/DINESH/.yosys_show.dot'.
Dumping module incomp_if2 to page 1.
Exec: { test -f '/home/DINESH/.yosys_show.dot.pid' && fuser -s '/home/DINESH/.yosys_show.dot.pid' && /dev/null; } || { echo \$\$ >&3; exec xdot '/home/DINESH/.yosys_show.dot' ; } > /home/DINESH/.yosys_show.dot.pid }

```
yosys>
```

incomp_if2

LAB: OPT_CHECK2

5.15.7. Executing MEMORY_SHARE pass (consolidating memories).

5.15.8. Executing OPT_MEM_WIDEN pass (optimize memory usage).
Performed a total of 0 transformations.

5.15.9. Executing OPT_CLEAN pass (remove unused cells or wires in module \opt_check2').
Finding unused cells or wires in module \opt_check2'.

5.15.10. Executing MEMORY_COLLECT pass (generating memory map).

5.16. Executing OPT_CLEAN pass (remove unused cells or wires in module \opt_check2').
Finding unused cells or wires in module \opt_check2'.

5.17. Executing OPT pass (performing simple optimizations).

5.17.1. Executing OPT_EXPR pass (perform constant folding).
Optimizing module opt_check2.
<suppressed -1 debug messages>

5.17.2. Executing OPT_MERGE pass (detect identical cells).
Finding identical cells in module \opt_check2'.
Removed a total of 0 cells.

5.17.3. Executing OPT_DFF pass (perform DFF optimizations).

5.17.4. Executing OPT_CLEAN pass (remove unused cells or wires in module \opt_check2').
Finding unused cells or wires in module \opt_check2'.

5.17.5. Finished Fast OPT passes.

5.18. Executing MEMORY_MAP pass (converting memories to logic and flip-flops).

LAB: MULTIPLE_MODULES

ABC: + &fraig -x
ABC: + &put
ABC: + &scorr
ABC: Warning: The network is combinational (run &dc2)
ABC: + &dc2
ABC: + &dretime
ABC: + &strash
ABC: + &get -n
ABC: + &dch -f
ABC: + &nf
ABC: + &put
ABC: + &write_blif <abc-temp-dir>/output.blif

5.3.2. Re-integrating ABC results.
ABC RESULTS: sky130_fd_sc_hd_or2_0 cells: 0
ABC RESULTS: internal signals: 0
ABC RESULTS: input signals: 2
ABC RESULTS: output signals: 1
Removing temp directory.

yosys> show

6. Generating Graphviz representation of design.
ERROR: For formats different than 'ps' or 'dot'

yosys> show multiple_modules

7. Generating Graphviz representation of design.
Writing dot description to /home/DINESH/.yosys-show.dot
Dumping module multiple_modules to page 1.
Exec: { test -f '/home/DINESH/.yosys-show.dot.p1' && cat /home/DINESH/.yosys-show.dot.p1 > /home/DINESH/.yosys-show.dot.pid' & }

LAB: PARTIAL_CASE_ASSIGN

File Machine View Input Devices Help

Sep 27 06:17

DINESH@UBUNTU: ~/Desktop/yosys

File Edit View Search Terminal Tabs Help

DINESH@UBUNTU: ~/Desktop/yosys

Yosys 0.33 (git sha1 2584903a060)

```

yosys> read_liberty -lib lib/sky130_fd_sc_hd_tt_025C_1v80.l
1. Executing Liberty frontend: lib/sky130_fd_sc_hd_tt_025C_1v80.l
Imported 418 cell types from liberty file.

yosys> read_verilog design/verilog_files/partial_case_assign.v
2. Executing Verilog-2005 frontend: design/verilog_files/partial_case_assign.v
Parsing Verilog input from 'design/verilog_files/partial_case_assign.v'
Generating RTLIL representation for module '\partial_case_assign'
Successfully finished Verilog frontend.

yosys> synth -top partial_case_assign
3. Executing SYNTH pass.
3.1. Executing HIERARCHY pass (managing design hierarchy).
3.1.1. Analyzing design hierarchy..
Top module: \partial_case_assign
3.1.2. Analyzing design hierarchy..
Top module: \partial_case_assign
Removed 0 unused modules.
3.2. Executing PROC pass (convert processes to netlists).
3.2.1. Executing PROC_CLEAN pass (remove empty switches from decision trees).
Cleaned up 0 empty switches.

```

yosys_showdot - Dot Viewer

LAB: TERNARY_OPERATOR_MUX

File Machine View Input Devices Help

Sep 27 05:27

DINESH@UBUNTU: ~/Desktop/yosys

File Edit View Search Terminal Tabs Help

DINESH@UBUNTU: ~/Desktop/yosys

```

3.6.7. Executing OPT_CLEAN pass (remove unused cells and wires).
Finding unused cells or wires in module \ternary_operator_mux.

3.6.8. Executing OPT_EXPR pass (perform const folding).
Optimizing module ternary_operator_mux.

3.6.9. Finished OPT passes. (There is nothing left to do.)

3.7. Executing FSM pass (extract and optimize FSM).
3.7.1. Executing FSM_DETECT pass (finding FSMs in design).
3.7.2. Executing FSM_EXTRACT pass (extracting FSM from design)
3.7.3. Executing FSM_OPT pass (simple optimizations of FSMs).
3.7.4. Executing OPT_CLEAN pass (remove unused cells and wires).
Finding unused cells or wires in module \ternary_operator_mux.
3.7.5. Executing FSM_OPT pass (simple optimizations of FSMs).
3.7.6. Executing FSM_RECODE pass (re-assigning FSM state encoding).
3.7.7. Executing FSM_INFO pass (dumping all available information).
3.7.8. Executing FSM_MAP pass (mapping FSMs to basic logic).

3.8. Executing OPT pass (performing simple optimizations).
3.8.1. Executing OPT_EXPR pass (perform const folding).
Optimizing module ternary_operator_mux.

```

yosys_showdot - Dot Viewer

LAB: DFF_SYNCRES

UBUNTU [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Sep 27 04:51

DINESH@UBUNTU: ~/Desktop/yosys

File Edit View Search Terminal Tabs Help

DINESH@UBUNTU: ~/Desktop/yosys

ABC: Memory = 19.52 MB. Time = 0.19 sec

ABC: Warning: Detected 9 multi-output gates (for

ABC: + strash

ABC: + &get -n

ABC: + &fraig -x

ABC: + &put

ABC: + scorr

ABC: Warning: The network is combinational (run

ABC: + dc2

ABC: + dretine

ABC: + strash

ABC: + &get -n

ABC: + &dch -f

ABC: + &nf

ABC: + &put

ABC: + write_blif <abc-temp-dir>/output.blif

5.1.2. Re-integrating ABC results.

ABC RESULTS: sky130_fd_sc_hd_nor2b_1 cells:

ABC RESULTS: internal signals: 1

ABC RESULTS: input signals: 2

ABC RESULTS: output signals: 1

Removing temp directory.

yosys> show

6. Generating Graphviz representation of design.

Writing dot description to '/home/DINESH/.yosys_s

Dumping module dff_syncres to page 1.

Exec: { test -f '/home/DINESH/.yosys_show.dot.pid

me/DINESH/.yosys_show.dot.pid' &

yosys>

yosys_show.dot - Dot Viewer

sync_reset

d

async_reset

BUF

A

B_N

sky130_fd_sc_hd_nor2b_1

Y

clk

CLK

D

sky130_fd_sc_hd_dfxtp_1

Q

q

dff_syncres

LAB: DFF_CONST1

UBUNTU [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Sep 27 05:06

DINESH@UBUNTU: ~/Desktop/yosys

File Edit View Search Terminal Tabs Help

DINESH@UBUNTU: ~/Desktop/yosys

ABC: Memory = 19.52 MB. Time = 0.13 sec

ABC: Warning: Detected 9 multi-output

ABC: + strash

ABC: + &get -n

ABC: + &fraig -x

ABC: + &put

ABC: + scorr

ABC: Warning: The network is combinat

ABC: + dc2

ABC: + dretine

ABC: + strash

ABC: + &get -n

ABC: + &dch -f

ABC: + &nf

ABC: + &put

ABC: + write_blif <abc-temp-dir>/out

6.1.2. Re-integrating ABC results.

ABC RESULTS: sky130_fd_sc_hd_clkiv

ABC RESULTS: internal signals

ABC RESULTS: input signals

ABC RESULTS: output signals

Removing temp directory.

yosys> show

7. Generating Graphviz representation

Writing dot description to '/home/DI

Dumping module dff_const1 to page 1.

Exec: { test -f '/home/DINESH/.yosys

me/DINESH/.yosys_show.dot.pid' &

yosys>

yosys_show.dot - Dot Viewer

reset

BUF

A

sky130_fd_sc_hd_clkinv_1

Y

clk

CLK

D

sky130_fd_sc_hd_dfrtp_1

Q

q

dff_const1

LAB: DFF_CONST3

ABC: Memory = 19.52 MB. Time = 0.20 sec
 ABC: Warning: Detected 9 multi-output
 ABC: + strash
 ABC: + &get -n
 ABC: + &fraig -x
 ABC: + &put
 ABC: + &corr
 ABC: Warning: The network is combinational (run
 ABC: + dc2
 ABC: + dretine
 ABC: + strash
 ABC: + &get -n
 ABC: + &dch -f
 ABC: + &nf
 ABC: + &put
 ABC: + write_blif <abc-temp-dir>/out
 5.1.2. Re-integrating ABC results.
 ABC RESULTS: sky130_fd_sc_hd_clkinv_1 cells:
 ABC RESULTS: internal signals: 0
 ABC RESULTS: input signals: 1
 ABC RESULTS: output signals: 1
 Removing temp directory.
 yosys> show
 6. Generating Graphviz representation
 Writing dot description to '/home/DINESH/.yosys_s
 Dumping module dff_const3 to page 1.
 Exec: { test -f '/home/DINESH/.yosys_s
 me/DINESH/.yosys_show.dot.pld' &
 yosys>

LAB: DFF_ASYNCRES

ABC: + strash
 ABC: + &get -n
 ABC: + &fraig -x
 ABC: + &put
 ABC: + &corr
 ABC: Warning: The network is combinational (run
 ABC: + dc2
 ABC: + dretine
 ABC: + strash
 ABC: + &get -n
 ABC: + &dch -f
 ABC: + &nf
 ABC: + &put
 ABC: + write_blif <abc-temp-dir>/output.blif
 7.1.2. Re-integrating ABC results.
 ABC RESULTS: sky130_fd_sc_hd_clkinv_1 cells:
 ABC RESULTS: internal signals: 0
 ABC RESULTS: input signals: 1
 ABC RESULTS: output signals: 1
 Removing temp directory.
 yosys> show
 8. Generating Graphviz representation of design.
 Writing dot description to '/home/DINESH/.yosys_s
 Dumping module dff_asyncres to page 1.
 Exec: { test -f '/home/DINESH/.yosys_s
 me/DINESH/.yosys_show.dot.pld' &
 yosys> /usr/lib/python3/dist-packages/xdot/elements.py:174: UserWarning: Font family 'Times-Roman' is not available, using 'Ubuntu Sans 11'
 warnings.warn(msg)
 yosys>

LAB: BAD_MUX

File Machine View Input Devices Help

Sep 27 05:27

DINESH@UBUNTU: ~/Desktop/yosys

File Edit View Search Terminal Tabs Help

DINESH@UBUNTU: ~/... x DINESH@UBUNTU: ~/... x DINESH@UBUNTU: ~/... x DINESH@UBUNTU: ~/... x DINESH@UBUNTU: ~/... x DINESH@UBUNTU: ~/... x DINESH@UBUNTU: ~/... x DINESH@UBUNTU: ~/... x

3.6.7. Executing OPT_CLEAN pass (remove unused cells and wires).
Finding unused cells or wires in module \ternary_operator_mux.

3.6.8. Executing OPT_EXPR pass (perform const folding).
Optimizing module ternary_operator_mux.

3.6.9. Finished OPT passes. (There is nothing left to do.)

3.7. Executing FSM pass (extract and optimize FSM).

3.7.1. Executing FSM_DETECT pass (finding FSMs in design).

3.7.2. Executing FSM_EXTRACT pass (extracting FSM from design).

3.7.3. Executing FSM_OPT pass (simple optimizations of FSMs).

3.7.4. Executing OPT_CLEAN pass (remove unused cells and wires).
Finding unused cells or wires in module \ternary_operator_mux.

3.7.5. Executing FSM_OPT pass (simple optimizations of FSMs).

3.7.6. Executing FSM_RECODE pass (re-assigning FSM state encod

3.7.7. Executing FSM_INFO pass (dumping all available informat

3.7.8. Executing FSM_MAP pass (mapping FSMs to basic logic).

3.8. Executing OPT pass (performing simple optimizations).

3.8.1. Executing OPT_EXPR pass (perform const folding).
Optimizing module ternary_operator_mux.

yosys_showdot - Dot Viewer

ternary_operator_mux